

UNIVERSIDAD NACIONAL DE CÓRDOBA

Facultad de Ciencias Exactas, Físicas y Naturales

Ingeniería en Computación

Trabajo Práctico N.º 3

Capas de enlace de datos, red y transporte

Asignatura: Redes de Computadoras

Grupo: MiLANesas

Alumnos: Tomas Brigido Kevin Cufre
Rocco Delgado Facundo Lugones
Javier Siñuka Agustin Tosolini

Fecha: Septiembre de 2026

Índice

1. Ítem 1: Organización de la información dentro de una red local	1
1.1. ¿Qué función cumple la capa de enlace dentro del modelo OSI? ¿Qué tipo de comunicación resuelve?	1
1.2. ¿Qué es una dirección MAC? ¿En qué se diferencia de una dirección IP? . .	1
1.3. ¿Qué es una trama Ethernet? Identificar sus principales campos y explicar brevemente para qué sirve cada uno.	2
1.4. ¿Qué información permite determinar qué protocolo de capa superior está transportando una trama Ethernet?	2
2. Ítem 2: Captura y análisis de tráfico con Wireshark	3
2.1. Seleccionar una trama Ethernet e identificar las direcciones MAC de origen y destino. ¿A qué dispositivos creen que corresponden?	3
2.2. Dentro de la misma trama, identificar el paquete IP. ¿Cuáles son las direcciones IP de origen y destino?	4
2.3. Comparar las direcciones MAC y las direcciones IP encontradas. ¿Representan lo mismo?	4
2.4. Observar el campo EtherType. ¿Qué protocolo está encapsulado dentro de la trama analizada?	5
3. Ítem 3: Transporte de información mediante TCP	6
3.1. ¿Qué problema(s) resuelve TCP que no resuelve directamente Ethernet ni IP? .	6
3.2. Investigar los campos más importantes de la metadata en un frame TCP. ¿Para qué sirve cada uno?	6
3.3. Explicar el Three-way y Four-way handshake en TCP.	7
3.4. Iniciar la conexión enviando un paquete, capturar el handshake y el paquete de datos. Analizar el paquete de datos, sus distintas partes y encontrar la carga útil del paquete usando Wireshark.	8
3.5. Finalizar la conexión y capturar el Four-way handshake.	10
3.6. ¿Qué conclusión podemos sacar de que sea tan fácil ver un paquete que viaja a través de la red?	10
4. Ítem 4: Comunicación con un servidor remoto mediante PacketSender	12
4.1. Iniciar una conexión con el servidor y obtener respuestas para los comandos admitidos: hola, ping, tic y status.	12
4.2. Iniciar la conexión, enviar el comando correspondiente al grupo y documentar la respuesta del servidor.	14

1. Ítem 1: Organización de la información dentro de una red local

1.1. ¿Qué función cumple la capa de enlace dentro del modelo OSI? ¿Qué tipo de comunicación resuelve?

La capa de enlace de datos (capa 2) es responsable de la **transferencia confiable de datos entre dos nodos directamente conectados** dentro de una misma red física. Sus funciones principales son:

- **Encapsular** los paquetes de la capa de red en tramas (*frames*).
- **Direccionamiento físico** mediante direcciones MAC.
- **Control de acceso al medio** (MAC): decide quién transmite cuándo cuando varios dispositivos comparten el medio.
- **Detección de errores** de transmisión mediante un código de verificación (FCS/CRC).

Resuelve la **comunicación nodo a nodo (*hop-to-hop*)** dentro del mismo segmento de red local, es decir, entre dispositivos que “se ven” directamente sin pasar por un router.

1.2. ¿Qué es una dirección MAC? ¿En qué se diferencia de una dirección IP?

Una **dirección MAC** (Media Access Control) es un identificador físico de 48 bits (6 bytes), habitualmente escrito en hexadecimal (ej: **98:f0:83:9e:9e:b1**), asignado de fábrica a cada interfaz de red. Es única a nivel mundial y, en principio, no cambia.

Diferencias principales con una dirección IP:

Característica	MAC	IP
Capa	2 (Enlace)	3 (Red)
Asignación	Fija, de fábrica	Lógica, configurable
Alcance	Red local (LAN)	Global (entre redes)
Función	Identifica el dispositivo físico	Identifica el nodo y permite el ruteo

En una comunicación, la IP indica **quién es el destino final**, mientras que la MAC indica **al siguiente dispositivo físico** al que hay que entregar la trama.

1.3. ¿Qué es una trama Ethernet? Identificar sus principales campos y explicar brevemente para qué sirve cada uno.

Una **trama Ethernet** es la unidad de datos de la capa de enlace en redes Ethernet. Encapsula un paquete de capa superior (típicamente IP) agregando información necesaria para su entrega dentro de la red local.

Campos principales:

- **Preámbulo (7 bytes) + SFD (1 byte):** sincronizan al receptor e indican el inicio de la trama.
- **MAC destino (6 bytes):** dirección física del receptor.
- **MAC origen (6 bytes):** dirección física del emisor.
- **EtherType / Length (2 bytes):** indica el protocolo de capa superior encapsulado (ej: 0x0800 para IPv4, 0x86DD para IPv6, 0x0806 para ARP).
- **Payload / Datos (46–1500 bytes):** contiene el paquete de la capa superior (ej: un paquete IP).
- **FCS (4 bytes):** *Frame Check Sequence*, un CRC que permite detectar errores en la trama recibida.



Figura 1: Estructura de una trama Ethernet.

1.4. ¿Qué información permite determinar qué protocolo de capa superior está transportando una trama Ethernet?

El campo **EtherType** de la trama Ethernet indica qué protocolo de capa superior se está transportando en el *payload*. Es un valor de 2 bytes que el receptor lee para saber a qué módulo entregar los datos desencapsulados. Ejemplos:

- 0x0800 → IPv4
- 0x86DD → IPv6
- 0x0806 → ARP
- 0x8100 → VLAN (802.1Q)

2. Ítem 2: Captura y análisis de tráfico con Wireshark

Para el desarrollo de esta actividad, se realizó una captura de tráfico de red en la interfaz local utilizando el software Wireshark. A partir de los paquetes capturados, se seleccionó para su análisis la trama identificada como **Frame 271**, correspondiente a un segmento de comunicación TCP entre el host local y un servidor remoto.

No.	Time	Source	Destination	Protocol	Length	Info
269	43.137058000	192.168.0.219	51.79.9.235	TCP	54	56154 → 16000 [ACK] Seq=9 Ack=888 Win=32694 Len=0
270	43.761793900	51.79.9.235	192.168.0.219	TCP	66	16000 → 56154 [PSH, ACK] Seq=888 Ack=9 Win=240 Len=12
271	43.888124700	192.168.0.219	51.79.9.235	TCP	54	56154 → 16000 [ACK] Seq=9 Ack=900 Win=32693 Len=0
272	43.839927700	192.168.0.219	57.144.207.32	TLSv1.2	124	Application Data
273	43.860832300	57.144.207.32	192.168.0.219	TCP	60	443 → 61902 [ACK] Seq=73 Ack=141 Win=1906 Len=0
274	43.997329000	57.144.207.32	192.168.0.219	TLSv1.2	126	Application Data
275	44.042206300	192.168.0.219	57.144.207.32	TCP	54	61902 → 443 [ACK] Seq=141 Ack=145 Win=255 Len=0
276	44.888136300	192.168.0.1	224.0.0.1	IGMPv3	60	Membership Query, general
277	44.962914400	192.168.0.219	172.217.117.4	TCP	55	[TCP Keep-Alive] 61455 → 443 [ACK] Seq=587 Ack=7174 Win=255 Len=1
278	44.985814900	172.217.117.4	192.168.0.219	TCP	66	[TCP Keep-Alive ACK] 443 → 61455 [ACK] Seq=7174 Ack=588 Win=1038 Len=0 SLE
279	45.554476100	51.79.9.235	192.168.0.219	TCP	66	16000 → 56154 [PSH, ACK] Seq=900 Ack=9 Win=240 Len=12
280	45.589853700	192.168.0.219	8.8.8.8	DNS	94	Standard query 0x9f76 A p2p-dfw2.discovery.steamserver.net
281	45.602988100	192.168.0.219	51.79.9.235	TCP	54	56154 → 16000 [ACK] Seq=9 Ack=912 Win=32693 Len=0
282	45.614938200	8.8.8.8	192.168.0.219	DNS	126	Standard query response 0x9f76 A p2p-dfw2.discovery.steamserver.net A 162.1
283	45.846068800	162.159.135.234	192.168.0.219	TLSv1.2	342	Application Data
284	45.896691400	192.168.0.219	162.159.135.234	TCP	54	63930 → 443 [ACK] Seq=122 Ack=1456 Win=511 Len=0
285	46.059071200	192.168.0.1	224.0.0.22	IGMPv3	70	Membership Report / Join group 224.0.0.22 for any sources / Join group 224
286	46.521531000	192.168.0.219	224.0.0.22	IGMPv3	54	Membership Report / Join group 224.0.0.252 for any sources
287	46.919283800	162.159.135.234	192.168.0.219	TLSv1.2	195	Application Data
288	46.971540700	192.168.0.219	162.159.135.234	TCP	54	63930 → 443 [ACK] Seq=122 Ack=1597 Win=511 Len=0
289	47.578602100	192.168.0.219	51.79.9.235	TCP	62	56154 → 16000 [PSH, ACK] Seq=9 Ack=912 Win=32693 Len=8
290	47.751714300	51.79.9.235	192.168.0.219	TCP	60	16000 → 56154 [ACK] Seq=912 Ack=17 Win=240 Len=0
291	48.178592900	51.79.9.235	192.168.0.219	TCP	62	16000 → 56154 [PSH, ACK] Seq=912 Ack=17 Win=240 Len=8
292	48.230630300	192.168.0.219	51.79.9.235	TCP	54	56154 → 16000 [ACK] Seq=17 Ack=920 Win=32692 Len=0
293	48.512527100	192.168.0.219	224.0.0.22	IGMPv3	54	Membership Report / Join group 239.255.255.250 for any sources
294	48.540725600	142.251.128.42	192.168.0.219	UDP	243	443 → 54467 Len=201
295	48.575246200	192.168.0.219	142.251.128.42	UDP	74	54467 → 443 Len=32
296	48.850002600	51.79.9.235	192.168.0.219	TCP	79	16000 → 56154 [PSH, ACK] Seq=920 Ack=17 Win=240 Len=25

▶ Frame 271: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device\NPF_{72B5A82-4385-4CCC-A182-9ED262AB003}, id 0

▼ Ethernet II, Src: ASUSTekCOMPU_13:dd:5b (a0:ad:9f:13:dd:5b), Dst: SagemcomBroa_98:26:47 (34:53:d2:98:26:47)

Destination: SagemcomBroa_98:26:47 (34:53:d2:98:26:47)

.....0..... = LG bit: Globally unique address (factory default)

.....0..... = IG bit: Individual address (unicast)

Source: ASUSTekCOMPU_13:dd:5b (a0:ad:9f:13:dd:5b)

.....0..... = LG bit: Globally unique address (factory default)

.....0..... = IG bit: Individual address (unicast)

Type: IPv4 (0x0800)

[Stream index: 0]

▶ Internet Protocol Version 4, Src: 192.168.0.219, Dst: 51.79.9.235

▶ Transmission Control Protocol, Src Port: 56154, Dst Port: 16000, Seq: 9, Ack: 900, Len: 0

Figura 2: Captura de tráfico en Wireshark y desglose de la trama Ethernet II (Frame 271).

2.1. Seleccionar una trama Ethernet e identificar las direcciones MAC de origen y destino. ¿A qué dispositivos creen que corresponden?

Al inspeccionar la cabecera Ethernet II de la trama seleccionada (Frame 271), se identifican los siguientes campos de direccionamiento físico:

- **Dirección MAC de Origen (Source MAC):** a0:ad:9f:13:dd:5b (etiquetada en la captura como ASUSTekCOMPU_13:dd:5b).
- **Dirección MAC de Destino (Destination MAC):** 34:53:d2:98:26:47 (etiquetada en la captura como SagemcomBroa_98:26:47).

Identificación de los dispositivos:

1. **Dispositivo de origen (a0:ad:9f:13:dd:5b):** Corresponde a la tarjeta de interfaz de red (NIC) de la computadora personal desde la cual se realizó la captura. El prefijo OUI (*Organizationally Unique Identifier*) a0:ad:9f identifica al fabricante

como **ASUSTeK Computer Inc.** (fabricante del motherboard / placa de red del equipo emisor).

2. **Dispositivo de destino (34:53:d2:98:26:47):** Corresponde al router hogareño o puerta de enlace predeterminada (*Default Gateway*) de la red local. El prefijo OUI 34:53:d2 pertenece al fabricante **Sagemcom Broadband SAS**, correspondiente al módem/router provisto por el proveedor de servicios de Internet (ISP) para la conexión a la red externa.

Es importante destacar que, debido a que el destinatario final del paquete se encuentra fuera de la red local (en Internet), la computadora no puede entregar la trama directamente a dicho servidor en la Capa 2. Por ello, la trama Ethernet tiene como destino local inmediato la dirección MAC del router, el cual se encargará de desencapsularla y encaminar el paquete IP hacia el siguiente salto.

2.2. Dentro de la misma trama, identificar el paquete IP. ¿Cuáles son las direcciones IP de origen y destino?

Desplegando la cabecera del protocolo de Capa de Red (*Internet Protocol Version 4*), se identifican las siguientes direcciones lógicas:

- **Dirección IP de Origen (Source IP):** 192.168.0.219
- **Dirección IP de Destino (Destination IP):** 51.79.9.235

La dirección 192.168.0.219 es una dirección IPv4 privada (perteneciente al rango RFC 1918 de Clase C) asignada dinámicamente por el servidor DHCP de la red local a la computadora emisora.

Por su parte, la dirección 51.79.9.235 es una dirección IPv4 pública y globalmente enrutable, correspondiente al servidor remoto en Internet con el cual se está manteniendo el intercambio de datos en el puerto TCP 16000.

2.3. Comparar las direcciones MAC y las direcciones IP encontradas. ¿Representan lo mismo?

No, no representan lo mismo. Ambas direcciones cumplen roles fundamentalmente distintos y operan en capas diferentes del modelo TCP/IP:

1. Capa de operación y alcance:

- La **dirección MAC** opera en la **Capa de Enlace de Datos (Capa 2)** y tiene un alcance puramente *local (hop-by-hop)*. Su función es permitir la entrega de la trama entre interfaces directamente conectadas dentro del mismo segmento físico de red. La dirección MAC de destino en este caso identifica al router local, ya que la trama física concluye su viaje allí.

- La **dirección IP** opera en la **Capa de Red (Capa 3)** y tiene un alcance *global extremo a extremo (end-to-end)*. Identifica la procedencia original y el destino final del paquete a través de toda la interred, manteniéndose invariable a lo largo del trayecto a través de múltiples routers intermediarios.

2. Naturaleza del direccionamiento:

- La dirección MAC es una dirección física plana de 48 bits fijada por el fabricante del hardware.
- La dirección IP es una dirección lógica jerárquica de 32 bits estructurada en red y host, permitiendo el enrutamiento y la división en subredes.

En síntesis, mientras que la dirección IP responde a la pregunta *¿cuál es el destino final de la comunicación en Internet?* (51.79.9.235), la dirección MAC responde a *¿a qué dispositivo físico local adyacente se le debe entregar la trama en el siguiente paso?* (34:53:d2:98:26:47, el router de la LAN).

2.4. Observar el campo EtherType. ¿Qué protocolo está encapsulado dentro de la trama analizada?

En el encabezado de la trama Ethernet II se encuentra el campo **Type (EtherType)** con el valor:

Type: 0x0800 (IPv4)

El valor hexadecimal 0x0800 es el identificador asignado por el estándar IEEE/IANA para indicar que la carga útil (*payload*) contenida dentro del campo de datos de la trama Ethernet corresponde a un paquete del protocolo **IPv4 (Internet Protocol version 4)**.

Este campo le indica a la capa de enlace receptora qué protocolo de capa superior (Capa 3) debe recibir y procesar los datos una vez que se remueve la cabecera Ethernet. A su vez, dentro del paquete IPv4 se encuentra el campo *Protocol: 6*, indicando que el protocolo de capa de transporte encapsulado es TCP.

3. Ítem 3: Transporte de información mediante TCP

3.1. ¿Qué problema(s) resuelve TCP que no resuelve directamente Ethernet ni IP?

Tanto Ethernet (capa de enlace) como IP (capa de red) ofrecen un servicio de entrega **no confiable y sin conexión**: los paquetes pueden perderse, duplicarse, llegar desordenados o corromperse, y ninguno de esos protocolos lo detecta ni lo corrige automáticamente.

TCP (Transmission Control Protocol), operando en la capa de transporte, resuelve los siguientes problemas:

- **Entrega confiable:** mediante números de secuencia y acuses de recibo (ACK), TCP detecta segmentos perdidos y los retransmite.
- **Orden de llegada:** los segmentos se reensamblan en el receptor en el orden correcto, independientemente del orden en que llegaron.
- **Control de flujo:** el receptor indica al emisor cuántos bytes puede aceptar (campo *Window Size*), evitando que lo desborde.
- **Control de congestión:** TCP adapta la tasa de envío cuando detecta congestión en la red (algoritmos como *slow start* y *congestion avoidance*).
- **Comunicación orientada a conexión:** establece y cierra sesiones de forma explícita (handshakes), lo que permite mantener estado en ambos extremos.
- **Detección de errores:** incluye un checksum obligatorio sobre la cabecera y los datos.

3.2. Investigar los campos más importantes de la metadata en un frame TCP. ¿Para qué sirve cada uno?

Un segmento TCP tiene una cabecera mínima de 20 bytes. Sus campos principales son:

Campo	Descripción
Puerto origen	Identifica el proceso emisor en el host origen (16 bits).
Puerto destino	Identifica el proceso receptor en el host destino (16 bits).
Número de secuencia	Indica el byte del flujo al que corresponde el primer byte de datos del segmento (32 bits).
Número de ACK	Indica el próximo byte que el receptor espera recibir; confirma la recepción de todos los bytes anteriores (32 bits).
Data offset	Longitud de la cabecera TCP en palabras de 32 bits; permite ubicar dónde comienzan los datos.
Flags	Bits de control: SYN (sincronización), ACK (confirmación), FIN (cierre), RST (reset), PSH (envío inmediato), URG (datos urgentes).
Window size	Cantidad de bytes que el receptor está dispuesto a aceptar sin ACK (control de flujo, 16 bits).
Checksum	Verificación de integridad de la cabecera y los datos (16 bits).
Urgent pointer	Indica el desplazamiento al último byte urgente; solo válido si el flag URG está activo.

Cuadro 1: Campos principales de la cabecera TCP.

3.3. Explicar el Three-way y Four-way handshake en TCP.

Three-way handshake (establecimiento de conexión):

El three-way handshake es el proceso mediante el cual cliente y servidor sincronizan sus números de secuencia iniciales y acuerdan los parámetros de la conexión antes de intercambiar datos:

1. **SYN:** el cliente envía un segmento con el flag SYN activo y un número de secuencia inicial aleatorio ($seq = x$).
2. **SYN-ACK:** el servidor responde con SYN y ACK activos, confirma el número del cliente ($ack = x + 1$) y anuncia su propio número de secuencia inicial ($seq = y$).
3. **ACK:** el cliente confirma el número del servidor ($ack = y + 1$). La conexión queda establecida y ambos lados pueden enviar datos.

Four-way handshake (cierre de conexión):

Como TCP es full-duplex, cada extremo debe cerrar su mitad de la conexión de forma independiente:

1. **FIN:** el extremo que desea cerrar (por ejemplo, el cliente) envía un segmento con el flag FIN activo.
2. **ACK:** el otro extremo (servidor) confirma el FIN. En este punto el servidor aún puede seguir enviando datos.
3. **FIN:** cuando el servidor también termina de enviar, envía su propio FIN.
4. **ACK:** el cliente confirma el FIN del servidor. La conexión queda completamente cerrada.

3.4. Iniciar la conexión enviando un paquete, capturar el handshake y el paquete de datos. Analizar el paquete de datos, sus distintas partes y encontrar la carga útil del paquete usando Wireshark.

Utilizando dos instancias de la herramienta **PacketSender** (una actuando como cliente y otra como servidor en la misma computadora), se estableció una conexión TCP por la interfaz de loopback (127.0.0.1) en el puerto 61361, capturando el tráfico generado con Wireshark.

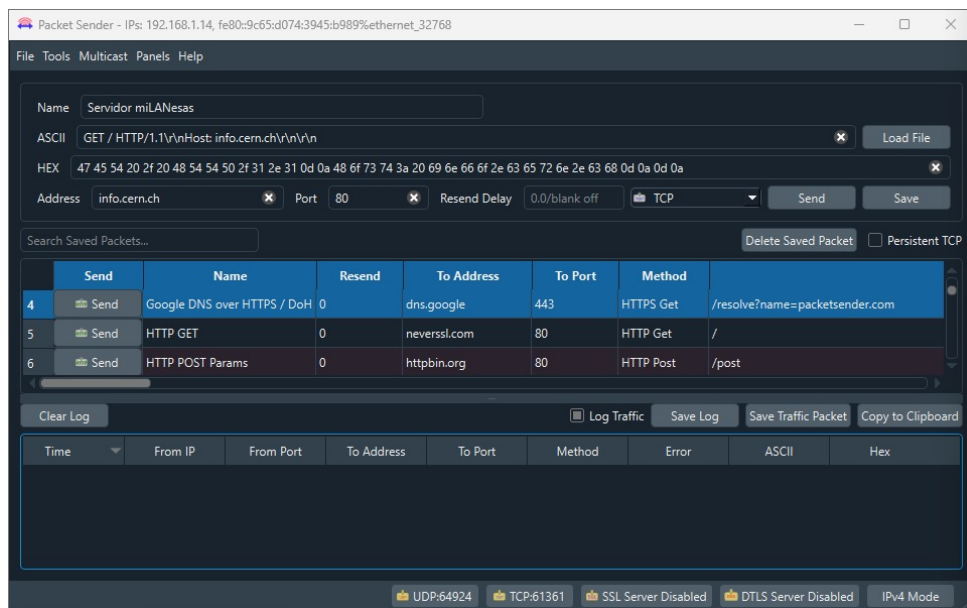


Figura 3: Configuración de PacketSender (instancia Servidor).

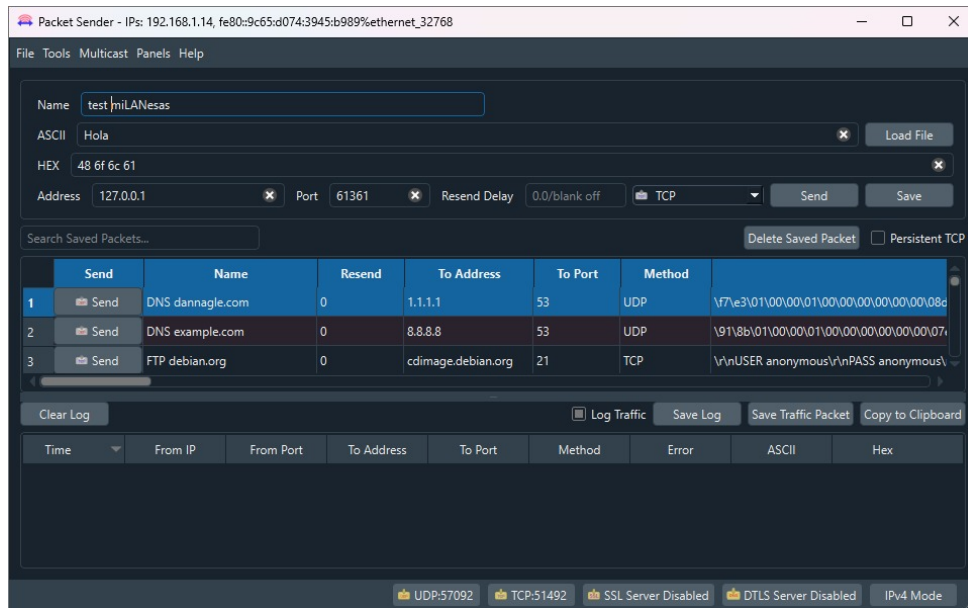


Figura 4: Configuración de PacketSender (instancia Cliente).

En primer lugar, se observa el *three-way handshake* mediante el cual se establece la conexión TCP:

No.	Time	Source	Destination	Protocol	Length	Info
10938	633.073525	127.0.0.1	127.0.0.1	TCP	56	57382 → 61361 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
10939	633.073639	127.0.0.1	127.0.0.1	TCP	56	61361 → 57382 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
10940	633.073682	127.0.0.1	127.0.0.1	TCP	44	57382 → 61361 [ACK] Seq=1 Ack=1 Win=65280 Len=0

Figura 5: Captura del Three-way handshake inicial en Wireshark (loopback).

Como se aprecia en la captura superior, la conexión se establece mediante tres paquetes fundamentales:

1. **Paquete SYN:** El cliente (puerto origen 57382) solicita abrir la conexión al servidor.
2. **Paquete SYN, ACK:** El servidor (puerto 61361) responde aceptando la conexión y sincronizando los números de secuencia.
3. **Paquete ACK:** El cliente confirma la recepción, quedando la conexión TCP establecida.

Inmediatamente después del establecimiento, se envía el paquete de datos (PSH, ACK) conteniendo el mensaje. A continuación se analiza en detalle este paquete y su respectiva confirmación:

10941	633.093199	127.0.0.1	127.0.0.1	TCP	48	57382 → 61361 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=4
Frame 10941: Packet, 48 bytes on wire (384 bits), 48 bytes captured (384 bits) on interface \Device\NPF_{...} id 0 Null/Loopback Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 Transmission Control Protocol, Src Port: 57382, Dst Port: 61361, Seq: 1, Ack: 1, Len: 4 Data (4 bytes) Data: 486f6c61 [Length: 4]						
				0000	02 00 00 00 45 00 00 2c b0 48 40 00 80 06 4c 81	...E...H...L
				0010	7f 00 00 01 7f 00 00 01 e0 26 ef b1 cd 24 f0 8e	&...\$...
				0020	5e bf 6c ec 50 18 00 ff a2 be 00 00 48 6f 6c 61	^..P...Holo

Figura 6: Análisis del paquete de datos y su carga útil (Data).

Al seleccionar este paquete de datos en Wireshark, identificamos sus distintas capas encapsuladas:

- **Capa de Enlace (Null/Loopback):** Al ser tráfico local, no viaja por Ethernet físico sino por una interfaz virtual de loopback.
- **Capa de Red (IPv4):** Indica que tanto la IP de origen como la de destino son la misma (127.0.0.1), confirmando que es tráfico interno.
- **Capa de Transporte (TCP):** Muestra el puerto de origen dinámico asignado al cliente (57382) y el puerto de destino del servidor PacketSender (61361).
- **Carga Útil (Payload):** Desplegando la sección *Data* al final de la pila de protocolos, encontramos la carga útil. Esta contiene la información transferida propiamente dicha, es decir, el texto o mensaje enviado desde el cliente hacia el servidor (por ejemplo, "Hola.º el nombre del grupo).

3.5. Finalizar la conexión y capturar el Four-way handshake.

Una vez que PacketSender finaliza la transmisión de datos, se produce el cierre ordenado de la conexión mediante el *four-way handshake*.

En la captura se pueden observar los paquetes de finalización al final del flujo, donde el equipo local y el servidor intercambian segmentos con las banderas FIN (indicando que ya no hay más datos para enviar) y ACK (confirmando la recepción de la solicitud de cierre del otro extremo), hasta que la conexión queda completamente cerrada en ambos extremos.

10943	633.094380	127.0.0.1	127.0.0.1	TCP	44 61361 → 57382 [FIN, ACK] Seq=1 Ack=5 Win=65280 Len=0
10944	633.094422	127.0.0.1	127.0.0.1	TCP	44 57382 → 61361 [ACK] Seq=5 Ack=2 Win=65280 Len=0
10945	633.094465	127.0.0.1	127.0.0.1	TCP	44 57382 → 61361 [FIN, ACK] Seq=5 Ack=2 Win=65280 Len=0
10946	633.094514	127.0.0.1	127.0.0.1	TCP	44 61361 → 57382 [ACK] Seq=2 Ack=6 Win=65280 Len=0

Figura 7: Captura del Four-way handshake cerrando la conexión TCP (paquetes con flag FIN y ACK).

3.6. ¿Qué conclusión podemos sacar de que sea tan fácil ver un paquete que viaja a través de la red?

El hecho de que herramientas como Wireshark permitan capturar e inspeccionar cualquier paquete que circula por la red local pone de manifiesto una limitación fundamental: **TCP/IP no cifra los datos por defecto**. La información viaja en texto plano y cualquier dispositivo en el mismo segmento de red puede interceptarla y leerla sin mayor dificultad.

Esto tiene implicancias importantes en materia de seguridad:

- Datos sensibles como contraseñas, cookies de sesión o información personal pueden ser capturados mediante ataques de tipo *sniffing* o *man-in-the-middle*.

- Protocolos como HTTP, FTP o Telnet son especialmente vulnerables ya que transmiten todo en texto plano.
- La solución adoptada por la industria es cifrar el contenido en la capa de aplicación mediante TLS/SSL (HTTPS, SFTP, SSH, etc.), de modo que aunque el atacante capture los paquetes, los datos sean ilegibles.

En conclusión, la facilidad de captura de tráfico demuestra que la seguridad en redes no puede depender únicamente del modelo TCP/IP, sino que requiere capas adicionales de cifrado y autenticación.

4. Ítem 4: Comunicación con un servidor remoto mediante PacketSender

4.1. Iniciar una conexión con el servidor y obtener respuestas para los comandos admitidos: hola, ping, tic y status.

Se configuró una instancia de PacketSender como cliente TCP apuntando a la dirección y puerto provistos por la cátedra para el servicio en la nube (34.136.251.235, puerto 5555). Se activaron las opciones **Persistent TCP**, para mantener la conexión abierta durante todo el intercambio, y **Append \r**, para que cada mensaje enviado incorporara automáticamente el terminador que el servidor espera al final de cada comando.

Una vez establecida la conexión se enviaron sucesivamente los comandos `hola`, `ping`, `tic` y `status`, obteniendo en cada caso una respuesta coherente por parte del servidor (un saludo, `pong`, `toc` y una descripción variable del estado del servicio, respectivamente). La Figura 8 muestra la ventana de la sesión persistente con el historial completo de comandos enviados y respuestas recibidas.

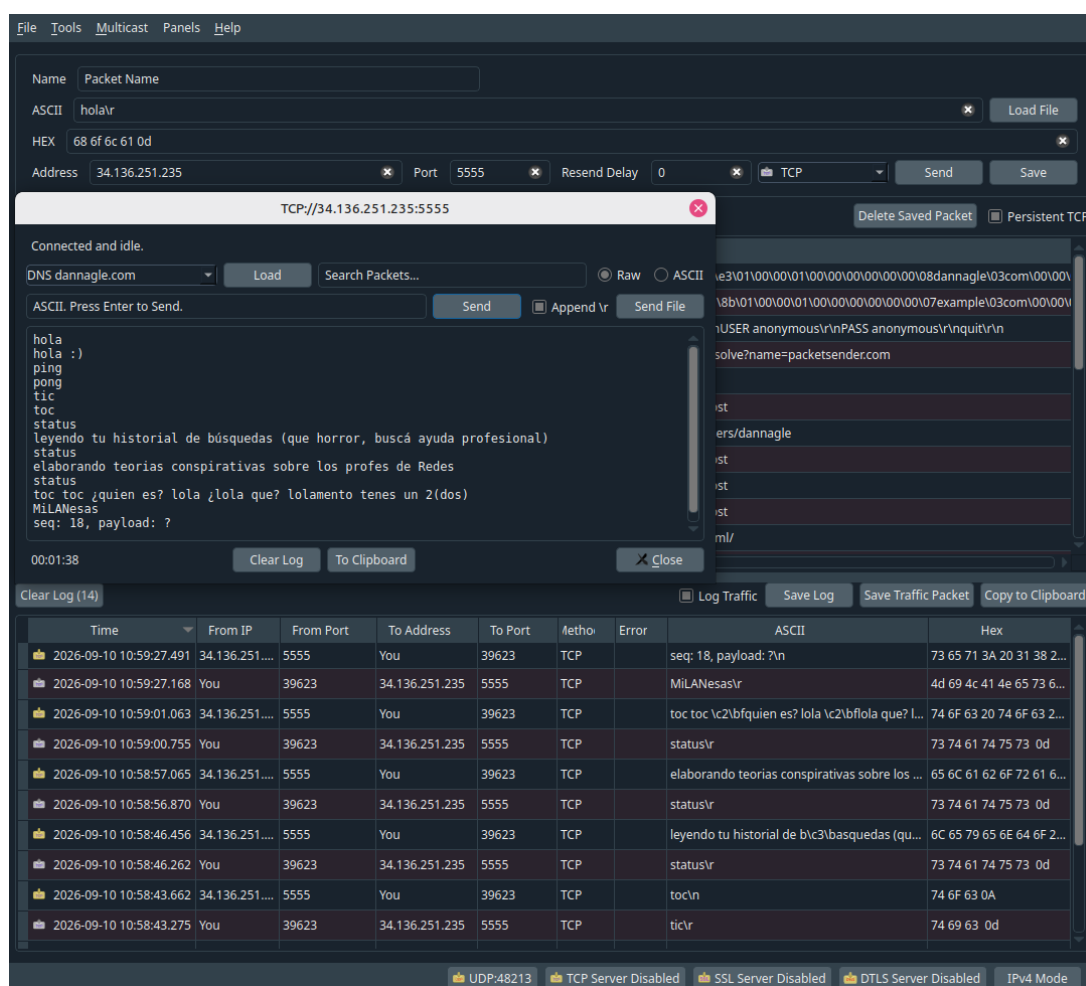


Figura 8: Sesión persistente de PacketSender con los comandos enviados al servidor y sus respuestas.

En paralelo se capturó la totalidad de la sesión con Wireshark, filtrando por `tcp.port == 5555` (Figura 9). La captura permite reconstruir con claridad todo el ciclo de vida de la conexión TCP, desde el establecimiento hasta el cierre.

No.	Time	Source	Destination	Protocol	Length	Info
13	2.879191438	192.168.0.97	34.136.251.235	TCP	74	39623 → 5555 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2278188347 TSecr=0 WS=1024
14	3.079216589	34.136.251.235	192.168.0.97	TCP	74	5555 → 39623 [SYN, ACK] Seq=0 Ack=1 Win=64768 Len=0 MSS=1420 SACK_PERM TSval=74298505 TSecr=2278188347 WS=128
15	3.079304485	192.168.0.97	34.136.251.235	TCP	66	39623 → 5555 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2278188538 TSecr=74298505
16	3.071291425	192.168.0.97	34.136.251.235	TCP	71	39623 → 5555 [PSH, ACK] Seq=1 Ack=1 Win=64512 Len=5 TSval=2278188539 TSecr=74298505
17	3.269364093	34.136.251.235	192.168.0.97	TCP	66	5555 → 39623 [ACK] Seq=1 Ack=6 Win=64768 Len=0 TSval=74298696 TSecr=2278188539
18	3.262231787	34.136.251.235	192.168.0.97	TCP	74	5555 → 39623 [PSH, ACK] Seq=1 Ack=6 Win=64768 Len=8 TSval=74298698 TSecr=2278188539
19	3.262219118	192.168.0.97	34.136.251.235	TCP	66	39623 → 5555 [ACK] Seq=6 Ack=9 Win=64512 Len=0 TSval=2278188738 TSecr=74298698
154	7.569556487	192.168.0.97	34.136.251.235	TCP	71	39623 → 5555 [PSH, ACK] Seq=6 Ack=9 Win=64512 Len=5 TSval=2278193837 TSecr=74298698
159	7.768862833	34.136.251.235	192.168.0.97	TCP	71	5555 → 39623 [PSH, ACK] Seq=9 Ack=11 Win=64768 Len=5 TSval=74303197 TSecr=2278193837
160	7.769922808	192.168.0.97	34.136.251.235	TCP	66	39623 → 5555 [ACK] Seq=11 Ack=14 Win=64512 Len=0 TSval=2278193229 TSecr=74303197
179	11.166528739	192.168.0.97	34.136.251.235	TCP	70	39623 → 5555 [PSH, ACK] Seq=11 Ack=14 Win=64512 Len=4 TSval=2278196634 TSecr=74303197
180	11.356795959	34.136.251.235	192.168.0.97	TCP	70	5555 → 39623 [PSH, ACK] Seq=14 Ack=15 Win=64768 Len=4 TSval=74306793 TSecr=2278196634
181	11.356857131	192.168.0.97	34.136.251.235	TCP	66	39623 → 5555 [ACK] Seq=15 Ack=18 Win=64512 Len=0 TSval=2278196824 TSecr=74306793
626	13.961256593	192.168.0.97	34.136.251.235	TCP	73	39623 → 5555 [PSH, ACK] Seq=15 Ack=18 Win=64512 Len=7 TSval=2278199429 TSecr=74306793
628	14.151128722	34.136.251.235	192.168.0.97	TCP	140	5555 → 39623 [PSH, ACK] Seq=18 Ack=22 Win=64768 Len=74 TSval=74309587 TSecr=2278199429
629	14.151187977	192.168.0.97	34.136.251.235	TCP	66	39623 → 5555 [ACK] Seq=22 Ack=92 Win=64512 Len=0 TSval=2278199619 TSecr=74309587
684	24.566889937	192.168.0.97	34.136.251.235	TCP	73	39623 → 5555 [PSH, ACK] Seq=22 Ack=92 Win=64512 Len=7 TSval=2278210035 TSecr=74309587
691	24.759349742	34.136.251.235	192.168.0.97	TCP	125	5555 → 39623 [PSH, ACK] Seq=92 Ack=29 Win=64768 Len=59 TSval=74320194 TSecr=2278210035
692	24.759408811	192.168.0.97	34.136.251.235	TCP	66	39623 → 5555 [ACK] Seq=29 Ack=151 Win=64512 Len=0 TSval=2278210227 TSecr=74320194
1170	28.565810365	192.168.0.97	34.136.251.235	TCP	73	39623 → 5555 [PSH, ACK] Seq=29 Ack=151 Win=64512 Len=7 TSval=2278214033 TSecr=74320194
1171	28.757225512	34.136.251.235	192.168.0.97	TCP	129	5555 → 39623 [PSH, ACK] Seq=151 Ack=36 Win=64768 Len=63 TSval=74324193 TSecr=2278214033
1172	28.757284446	192.168.0.97	34.136.251.235	TCP	66	39623 → 5555 [ACK] Seq=36 Ack=214 Win=64512 Len=0 TSval=2278214225 TSecr=74324193
1735	54.993610976	192.168.0.97	34.136.251.235	TCP	76	39623 → 5555 [PSH, ACK] Seq=36 Ack=214 Win=64512 Len=10 TSval=2278240461 TSecr=74324193
1738	55.186806785	34.136.251.235	192.168.0.97	TCP	86	5555 → 39623 [PSH, ACK] Seq=214 Ack=46 Win=64768 Len=20 TSval=74350621 TSecr=2278240461
1739	55.186967481	192.168.0.97	34.136.251.235	TCP	66	39623 → 5555 [ACK] Seq=46 Ack=234 Win=64512 Len=0 TSval=2278240654 TSecr=74350621
3418	124.0769436	192.168.0.97	34.136.251.235	TCP	66	39623 → 5555 [FIN, ACK] Seq=46 Ack=234 Win=64512 Len=0 TSval=2278389545 TSecr=74350621
3419	124.2679485	34.136.251.235	192.168.0.97	TCP	66	5555 → 39623 [FIN, ACK] Seq=234 Ack=47 Win=64768 Len=0 TSval=74419704 TSecr=2278389545
3420	124.2679753	192.168.0.97	34.136.251.235	TCP	66	39623 → 5555 [ACK] Seq=47 Ack=235 Win=64512 Len=0 TSval=2278389736 TSecr=74419704

Figura 9: Captura de Wireshark de la sesión TCP completa contra el servidor remoto.

Lo primero que se observa es el three-way handshake: el cliente (192.168.0.97, puerto efímero 39623) abre la conexión con un segmento **SYN**, cuyo número de secuencia relativo es **Seq=0** (Wireshark expresa los números de secuencia en forma relativa al inicio de la conexión, no como el valor absoluto de 32 bits que realmente viaja en el paquete). El servidor responde con **SYN, ACK**, con su propio **Seq=0** y **Ack=1**: aunque el segmento **SYN** no transporta datos, consume igualmente un número de secuencia, por lo que el servidor confirma esperando a partir del byte 1. El cliente cierra el handshake con un **ACK** puro (**Seq=1 Ack=1**), quedando la conexión establecida.

A partir de ese punto, cada segmento con datos (marcado **PSH, ACK**) hace avanzar el número de secuencia de su emisor en tantas unidades como bytes de carga útil transporte (campo **Len**), y el otro extremo confirma la recepción con un **ACK** cuyo valor es exactamente **Seq + Len** recibido, es decir, el número del *siguiente* byte que espera recibir. Por ejemplo, cuando el cliente envía `hola\r` (**Seq=1, Len=5**), el servidor confirma con **Ack=6**; cuando a continuación el servidor responde con 8 bytes de payload (**Seq=1, Len=8**), el cliente reconoce con **Ack=9**. Este patrón se repite en cada intercambio de la sesión —ping, tic, status y los mensajes adicionales enviados durante la prueba—, de modo que los contadores de secuencia de cliente y servidor crecen de forma monótona e independiente entre sí, reflejando en todo momento cuántos bytes efectivos ha transmitido cada extremo.

Finalmente, la conexión se cierra mediante un four-way handshake: el cliente envía **FIN, ACK** con **Seq=46 Ack=234**; el servidor responde con **FIN, ACK** con **Seq=234 Ack=47**, confirmando el **FIN** del cliente y cerrando su propio extremo en el mismo segmento (por eso en la captura se observan solo tres paquetes de cierre en lugar de cuatro: el **ACK** y el **FIN** del servidor viajan juntos); y el cliente confirma con un **ACK** final, **Seq=47 Ack=235**. Igual que ocurrió con el **SYN** inicial, la bandera **FIN** también consume un número de secuencia, motivo por el cual, nuevamente, el **ACK** de confirmación resulta ser exactamente uno más que el **Seq** del **FIN** recibido.

4.2. Iniciar la conexión, enviar el comando correspondiente al grupo y documentar la respuesta del servidor.

El servidor está configurado para reconocer, además de los comandos genéricos, un comando particular por cada grupo de la cátedra correspondiente a su nombre. Sobre la misma conexión persistente se envió el comando `MiLANesas\r`, obteniendo como respuesta `seq: 18, payload: ?` (Figura 10), lo que confirma que el servidor identificó correctamente al grupo. Esta respuesta, junto con el resto de la interacción, fue documentada además en la pestaña correspondiente de la planilla compartida de la cátedra.

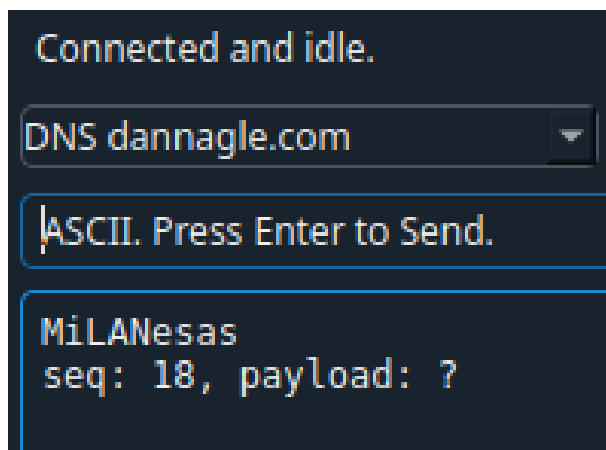


Figura 10: Respuesta del servidor al enviar el comando correspondiente al nombre del grupo.